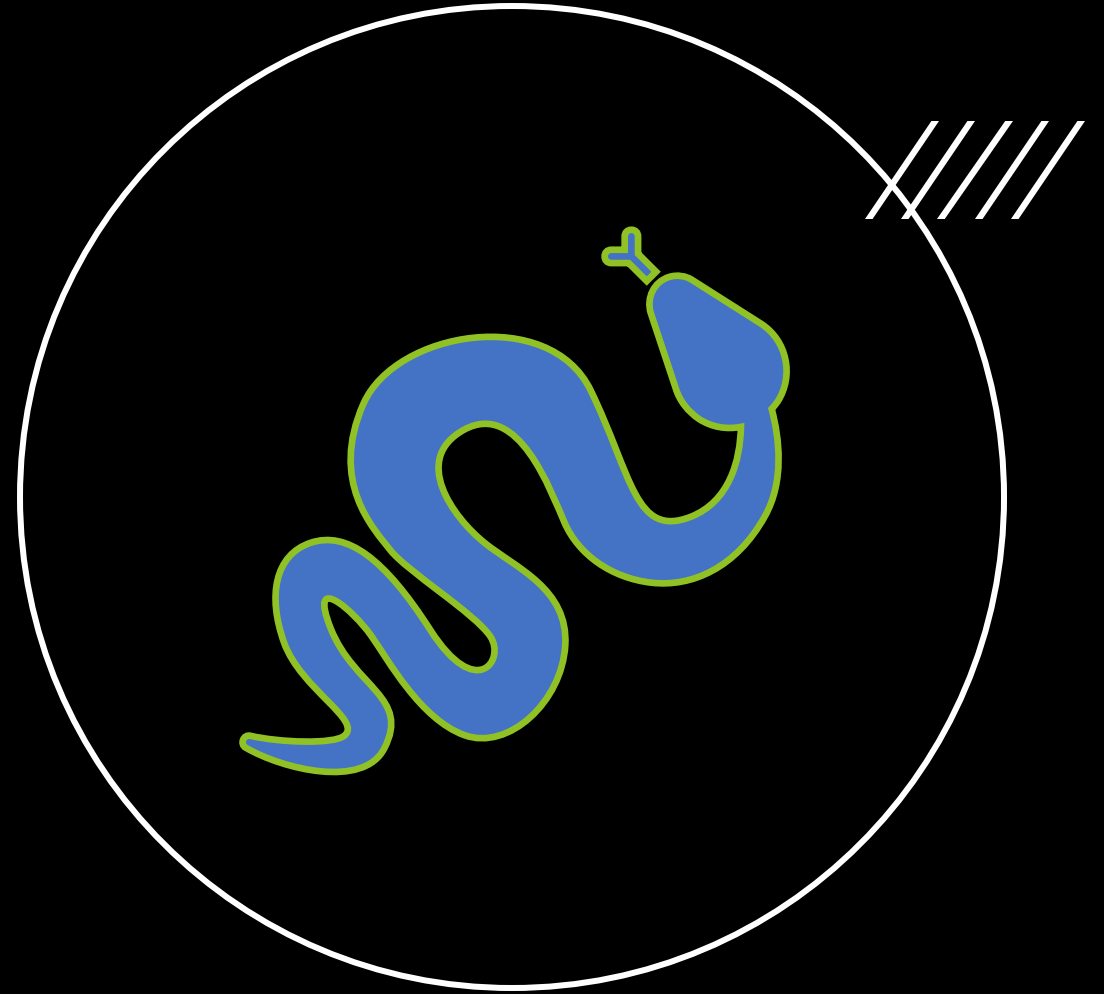


OOP & Python



Class definition

- Classes are defined using the *class* keyword with a very familiar structure:

```
class ClassName:  
    <statement-1>  
    . . .  
    <statement-N>
```

- There is no notion of a header file to include so we don't need to break up the creation of a class into declaration and definition. We just declare and use it!

Class definition

- Statements are usually assignments or function definitions
- Entering a class definition creates a new "namespace"-ish
- Really, a special `__dict__` attribute where attributes live
- Exiting a class definition creates a class object

Class objects

- Let's say I have a simple class which does not much of anything at all.

```
class MyClass:  
    """A simple example class docstring"""  
    i = 12345  
    def f(self):  
        return 'hello world'
```

Attribute References

MyClass.i # => 12345 (int object)

MyClass.f # => <function f> (function object)

Class instantiation

- I can create a new instance of MyClass using the familiar function notation.

```
x = MyClass()
```

```
float('3.5')
```

```
int(101001, base=2)
```

```
str(41)
```

```
list(('hap.py'))
```

```
dict(a=1, b=2)
```

Class Objects

- I can access the attributes and methods of my object in the following way:

```
>>> num = x.i  
>>> x.f()
```

Custom Constructor using `__init__`

We can define the special method `__init__()` which is automatically invoked for new instances (constructor).

```
class MyClass:
    """A simple example class"""
    i = 12345
    def __init__(self):
        print ("I just created a MyClass object!")
    def f(self):
        return 'hello world'
```

Instance Attribute Reference Resolution

```
class MyOtherClass:  
    num = 12345  
    def __init__(self):  
        self.num = 0
```

```
x = MyOtherClass()  
print(x.num) # 0 or 12345?  
del x.num  
print(x.num) # 0 or 12345?
```

Custom Constructor using `__init__`

```
class Complex:
    def __init__(self, realpart=0,
imagpart=0):
        self.real = realpart
```

```
# Make an instance object!
c = Complex(3.0, -4.5)
c.real, c.imag # => (3.0, -4.5)
```

Data attributes

- Like local variables in Python, there is no need for a data attribute to be declared before use.

```
>>> class Complex:
...     def __init__(self, realpart, imagpart):
...         self.r = realpart
...         self.i = imagpart
>>> x = Complex(3.0, -4.5)
>>> x.r, x.i
(3.0, -4.5)
>>> x.r_squared = x.r**2
>>> x.r_squared
9.0
```

Data Attributes

- We can add, modify, or delete attributes at will.

```
x.year = 2014 # Add an 'year' attribute.  
x.year = 2015 # Modify 'year' attribute.  
del x.year    # Delete 'year' attribute.
```

- There are also some built-in functions we can use to accomplish the same tasks.

```
hasattr(x, 'year')    # Returns true if year attribute exists  
getattr(x, 'year')    # Returns value of year attribute  
setattr(x, 'year', 2015) # Set attribute year to 2015  
delattr(x, 'year')    # Delete attribute year
```

Variables within classes

- Generally speaking, variables in a class fall under one of two categories:
 - **Class variables**, which are shared by all instances.
 - **Instance variables**, which are unique to a specific instance.

```
>>> class Dog:
...     kind = 'canine' # class var
...     def __init__(self, name):
...         self.name = name # instance var
>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.kind # shared by all dogs

>>> e.kind # shared by all dogs

>>> d.name # unique to d

>>> e.name # unique to e
```

Variables within classes

- Be careful when using mutable objects as class variables.

```
>>> class Dog:
>>>     tricks = [] # mutable class variable
>>>     def __init__(self, name):
>>>         self.name = name
>>>     def add_trick(self, trick):
>>>         self.tricks.append(trick)
>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.add_trick('roll over')
>>> e.add_trick('play dead')
>>> d.tricks # unexpectedly shared by all
```

Variables within classes

- To fix this issue, make it an instance variable instead.

```
>>> class Dog:
>>>     def __init__(self, name):
>>>         self.name = name
>>>         self.tricks = []
>>>     def add_trick(self, trick):
>>>         self.tricks.append(trick)
>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.add_trick('roll over')
>>> e.add_trick('play dead')
>>> d.tricks
['roll over']
>>> e.tricks
['play dead']
```

Built-in Attributes

Besides the class and instance attributes, every class has access to the following:

- **`__dict__`**: dictionary containing the object's namespace.
- **`__doc__`**: class documentation string or None if undefined.
- **`__name__`**: class name.
- **`__module__`**: module name in which the class is defined. This attribute is `"__main__"` in interactive mode.
- **`__bases__`**: a possibly empty tuple containing the base classes, in the order of their occurrence in the base class list.

Methods

- We can call a method of a class object using the familiar function call notation.
- Perhaps you noticed, however, that the definition of `MyClass.f()` involves an argument called *self*.

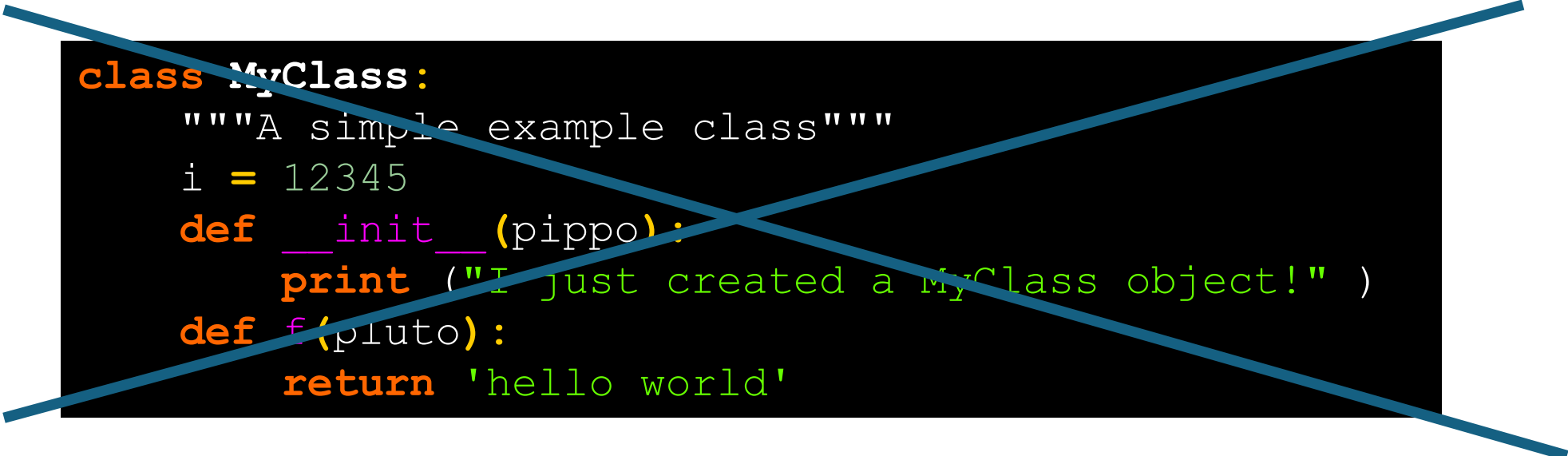
```
>>> x = MyClass()  
>>> x.f()  
'hello world'
```

Calling `x.f()` is
equivalent
to calling
`MyClass.f(x)`.

```
class MyClass:  
    """A simple example class"""  
    i = 12345  
    def __init__(self):  
        print ("I just created a MyClass object!" )  
    def f(self):  
        return 'hello world'
```

Methods

```
class MyClass:
    """A simple example class"""
    i = 12345
    def __init__(pippo):
        print ("I just created a MyClass object!" )
    def f(pippo):
        return 'hello world'
```



```
class MyClass:
    """A simple example class"""
    i = 12345
    def __init__(pippo):
        print ("I just created a MyClass object!" )
    def f(pluto):
        return 'hello world'
```

Method

```
class prova:  
  
    def f(self):  
  
        print("sono f")  
  
  
x = prova()  
  
x.f = lambda x, y: x + y
```

Class method

```
class MiaClasse:  
    @classmethod  
    def mio_metodo(cls):  
        print("Sono un metodo di classe!")  
        print("La classe è:", cls)
```

MiaClasse.mio_metodo()

```
class Persona:  
    def __init__(self, nome, eta):  
        self.nome = nome  
        self.eta = eta
```

```
@classmethod  
def da_stringa(cls, stringa):  
    nome, eta = stringa.split("-")  
    return cls(nome, int(eta))
```

```
p = Persona.da_stringa("Luca-30")
```

```
class Contatore:  
    count = 0  
  
    @classmethod  
    def incrementa(cls):  
        cls.count += 1
```

STATIC METHOD

```
class A:  
    def metodo_normale(self):  
        pass
```

```
@staticmethod  
def metodo_statico():  
    pass
```

Example

```
class Matematica:
```

```
    @staticmethod
```

```
    def add(a, b):
```

```
        return a + b
```

Fraction Example

```
>>> import frac
>>> f1 = frac.Fraction()
>>> f2 = frac.Fraction(3,5)
>>> f1.get_numerator()
0
>>> f1.get_denominator()
1
>>> f2.get_numerator()
3
>>> f2.get_denominator()
5
```

Fraction example

```
>>> f2.evaluate()  
0.6  
>>> f1.set_value(2,7)  
>>> f1.evaluate()  
0.2857142857142857  
>>> f1.show()  
2/7  
>>> f2.show()  
3/5  
>>> f2.input()  
2/3  
>>> f2.show()  
2/3
```

Pet example

```
class Pet:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def get_name(self):
        return self.name
    def get_age(self):
        return self.age
    def __str__(self):
        return "This pet's name is " + str(self.name)
```

The `__str__` built-in function defines what happens when I print an instance of `Pet`. Here I'm overriding it to print the name.

Pet example

```
>>> from pet import Pet
>>> mypet = Pet('Ben', '1')
>>> print mypet
This pet's name is Ben
>>> mypet.get_name()
'Ben'
>>> mypet.get_age()
```

```
class Pet:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def get_name(self):
        return self.name
    def get_age(self):
        return self.age
    def __str__(self):
        return "This pet's name is " + str(self.name)
```

Methods overload

Python does not support method overloading in the same way that some other programming languages, like Java or C++, do.

you can define multiple functions or methods with the same name in the same class or module, but only the latest defined method with that name will be kept, effectively overwriting the previous ones.

Python does not provide built-in method overloading based on the number or types of arguments.

However, Python does provide a degree of flexibility when it comes to function and method arguments. You can use default arguments and variable-length argument lists to achieve similar functionality to method overloading, albeit in a different way.

```
class Pet:
    def __init__(self, name):
        self.name = name

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
P = Pet("Cassie", 8) #ok
```

```
P = Pet("Cassie") #error
```

```
TypeError Traceback (most recent call last) Cell In[81], line 1
----> 1 P= Pet("Cassie") TypeError: __init__() missing 1
required positional argument: 'age'
```

```
class Pet:
```

```
    def __init__(self, name, age=None):  
        self.name = name  
        self.age = age
```

```
P = Pet("Cassie", 8) #ok
```

```
P = Pet("Cassie") #ok
```

```
class MyClass:
```

```
    i = 12345
```

```
    def __init__(self,x=None):
```

```
        print ("I just created a MyClass object!")
```

```
    def f(self,x):
```

```
        if x == None:
```

```
            return 'ciao'
```

```
        else:
```

```
            return "ciao " + x
```

inheritance

```
class DerivedClassName(BaseClassName):  
    <statement-1>  
    ...  
    <statement-N>
```

In the case of BaseClass being defined elsewhere, you can use `module_name.BaseClassName`.

inheritance

A class object 'remembers' its base class

All class objects inherit from object (default in Python 3)

All attribute references start from derived class

 This includes methods, as attributes of the class object!

Proceeds up the chain of base classes

Derived methods override (shadow) base methods

Similar to virtual in C++

inheritance

- Here is an example definition of a Dog class which inherits from Pet.

```
class Dog(Pet):  
    pass
```

- The pass statement is only included here for syntax reasons. This class definition for Dog essentially makes Dog an alias for Pet.

inheritance

- We've inherited all the functionality of our Pet class, now let's make the Dog class more interesting.

```
>>> from dog import Dog
>>> mydog = Dog('Ben', 1)
>>> print mydog
This pet's name is Ben
>>> mydog.get_name()
'Ben'
>>> mydog.get_age()
```

```
class Dog(Pet):
    pass
```

inheritance

- For my Dog class, I want all of the functionality of the Pet class with one extra attribute: breed. I also want some extra methods for accessing this attribute.

```
class Dog(Pet):  
    def __init__(self, name, age, breed):  
        Pet.__init__(self, name, age)  
        self.breed = breed  
    def get_breed(self):  
        return self.breed
```

inheritance

- For my Dog class, I want all of the functionality of the Pet class with one extra attribute: breed. I also want some extra methods for accessing this attribute.

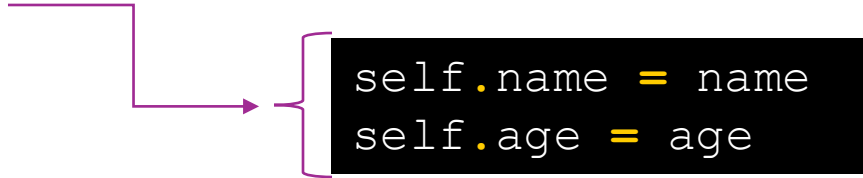
Overriding initialization function

```
class Dog(Pet):  
    def __init__(self, name, age, breed):  
        Pet.__init__(self, name, age)  
        self.breed = breed  
    def get_breed(self):  
        return self.breed
```

Python resolves attribute and method references by first searching the derived class and then searching the base class.

inheritance

```
class Dog(Pet):  
    def __init__(self, name, age, breed):  
        Pet.__init__(self, name, age)  
        self.breed = breed  
    def get_breed(self):  
        return self.breed
```



```
self.name = name  
self.age = age
```

We can call base class methods directly using `BaseClassName.method(self, arguments)`. Note that we do this here to extend the functionality of Pet's initialization method.

inheritance

```
>>> from dog import Dog
>>> mydog = Dog('Ben', 1, 'Maltese')
>>> print (mydog)
This pet's name is Ben
>>> mydog.get_age()
1
>>> mydog.get_breed()
'Maltese'
```

```
class Dog(Pet):
    def __init__(self, name, age, breed):
        Pet.__init__(self, name,
age)
        self.breed = breed
    def get_breed(self):
        return self.breed
```

inheritance

- Python has two notable built-in functions:
- `isinstance(object, classinfo)` returns true if *object* is an instance of *classinfo* (or some class derived from *classinfo*).
- `issubclass(class, classinfo)` returns true if *class* is a subclass of *classinfo*.

```
>>> from pet import Pet
>>> from dog import Dog
>>> mydog = Dog('Ben', 1, 'Maltese')
>>> isinstance(mydog, Dog)
True
>>> isinstance(mydog, Pet)
True
>>> issubclass(Dog, Pet)
True
>>> issubclass(Pet, Dog)
False
```

MULTIPLE INHERITANCE

Attribute resolution is performed by searching DerivedClassName, then Base1, then Base2, etc.

```
class DerivedClassName (Base1, Base2, Base3):  
    <statement-1>  
    ...  
    <statement-N>
```

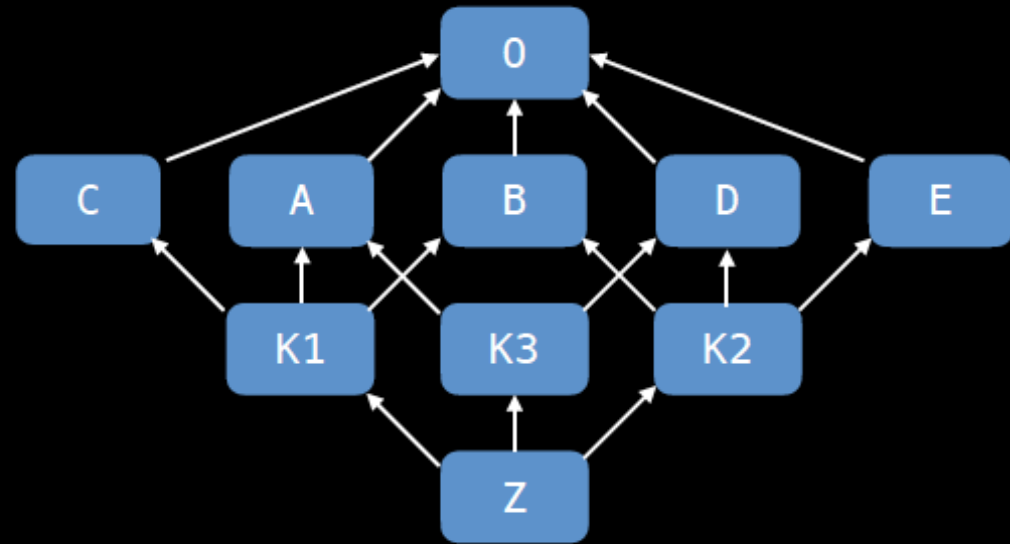
Attribute lookup is (almost) breadth-first, left-to-right
Class objects have a (hidden) function attribute .mro()
Shows linearization of base classes

```
bool.mro() # => [bool, int, object]
```

MULTIPLE INHERITANCE: an example of linearization

```
class A: pass
class B: pass
class C: pass
class D: pass
class E: pass
class K1(A, B, C): pass
class K2(D, B, E): pass
class K3(D, A): pass
class Z(K1, K2, K3): pass
```

```
Z.mro() # [Z, K1, K2, K3, D, A, B, C, E, object]
```



Operator Overloading

- *Overloading* means giving an operator (or function) a definition that depends on context
 - Example: a/b means something different depending on whether a and b are `ints` or `floats`
- Some languages (including Python) let users overload operators at will
- $y = a * x * x + b * x + c$

Overloading in Python

- Most Python operators can be overloaded
 - Just define a special function in a class
 - Annoying exception: plain assignment
- Many finicky rules
 - Names aren't always obvious
 - Some operators have “forward” and “reverse” versions
 - Some operators derive from others by default

Basic Overloading

```
class MyMath:
    def __init__(self, ...): ...
    def __add__(self, rhs):
        # Only works if rhs is a MyMath
        result = self.value + rhs.value
        return result
    def __sub__(self, rhs): ...
    # etc.
```

BASIC OVERLOADING

binary

+ __add__(self, other)	< __LT__(SELF, OTHER)	-= __ISUB__(SELF, OTHER)
- __sub__(self, other)	> __GT__(SELF, OTHER)	+= __IADD__(SELF, OTHER)
* __mul__(self, other)	<= __LE__(SELF, OTHER)	*= __IMUL__(SELF, OTHER)
/ __truediv__(self, other)	>= __GE__(SELF, OTHER)	/= __IDIV__(SELF, OTHER)
// __floordiv__(self, other)	== __EQ__(SELF, OTHER)	//= __IFLOORDIV__(SELF,
% __mod__(self, other)	!= __NE__(SELF, OTHER)	OTHER)
** __pow__(self, other)	< __LT__(SELF, OTHER)	%= __IMOD__(SELF, OTHER)
>> __rshift__(self, other)	> __GT__(SELF, OTHER)	**= __IPOW__(SELF, OTHER)
<< __lshift__(self, other)	<= __LE__(SELF, OTHER)	>>= __IRSHIFT__(SELF, OTHER)
& __and__(self, other)	>= __GE__(SELF, OTHER)	<<= __ILSHIFT__(SELF, OTHER)
__or__(self, other)	== __EQ__(SELF, OTHER)	&= __IAND__(SELF, OTHER)
^ __xor__(self, other)	!= __NE__(SELF, OTHER)	= __IOR__(SELF, OTHER)
		^= __IXOR__(SELF, OTHER)

unary

- __NEG__(SELF, OTHER)
+ __POS__(SELF, OTHER)
~ __INVERT__(SELF, OTHE

`__init__` method

- When a class inherits from another class, the `__init__` method of the base class (superclass) is not automatically called for objects of the subclass.
- you can explicitly call the base class's `__init__` method if needed, typically using the `super()` function.

Here's how it works:

- When you create an instance of a subclass, Python will call the `__init__` method of the subclass if it's defined in the subclass. If the `__init__` method is not defined in the subclass, Python will look up the class hierarchy to find it in a superclass.
- If the `__init__` method is defined in the subclass and you want to call the superclass's `__init__`, you can use the `super()` function. This allows you to call the superclass's `__init__` method before or after performing the subclass-specific initialization.

Example

```
class Parent:
    def __init__(self, name):
        self.name = name

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name) # Call the Parent's __init__
        self.age = age

child = Child("Alice", 5)
print(child.name) # Output: Alice
print(child.age) # Output: 5
```

Private variables

- There is no strict notion of a private attribute in Python.
- However, if an attribute is prefixed with a single underscore (e.g. `_name`), then it should be treated as private. Basically, using it should be considered bad form as it is an implementation detail.
- To avoid complications that arise from overriding attributes, Python does perform *name mangling*. Any attribute prefixed with two underscores (e.g. `__name`) and no more than one trailing underscore is automatically replaced with `_classname__name`.
- *Bottom line*: if you want others developers to treat it as private, use the appropriate prefix.

Name mangling

```
class Mapping:
    def __init__(self, iterable):
        self.items_list = []
        self.update(iterable)
    def update(self, iterable):
        for item in iterable:
            self.items_list.append(item)

class MappingSubclass(Mapping):
    def update(self, keys, values):
        for item in zip(keys, values):
            self.items_list.append(item)
```

What's the problem here?

Name mangling

```
class Mapping:
    def __init__(self, iterable):
        self.items_list = []
        self.update(iterable)
    def update(self, iterable):
        for item in iterable:
            self.items_list.append(item)

class MappingSubclass(Mapping):
    def update(self, keys, values):
        for item in zip(keys, values):
            self.items_list.append(item)
```

What's the problem here?

The update method of Mapping accepts one iterable object as an argument.

The update method of MappingSubclass, however, accepts keys and values as arguments.

Because MappingSubclass is derived from Mapping and we haven't overridden the `__init__` method, we will have an error when the `__init__` method calls update with a single argument.

Name mangling

To be clearer, because MappingSubclass inherits from Mapping but does not provide a definition for `__init__`, we implicitly have the following

```
class Mapping:
    def __init__(self, iterable):
        self.items_list = []
        self.update(iterable)
    def update(self, iterable):
        for item in iterable:
            self.items_list.append(item)
```

```
class MappingSubclass(Mapping):
    def update(self, keys, values):
        for item in zip(keys, values):
            self.items_list.append(item)
```

A blue arrow points from the `Mapping` class name in the `MappingSubclass` definition to the `Mapping` class definition above. A blue bracket on the right side of the `MappingSubclass` definition indicates that it inherits from `Mapping`.

```
def __init__(self, iterable):
    self.items_list = []
    self.update(iterable)
```

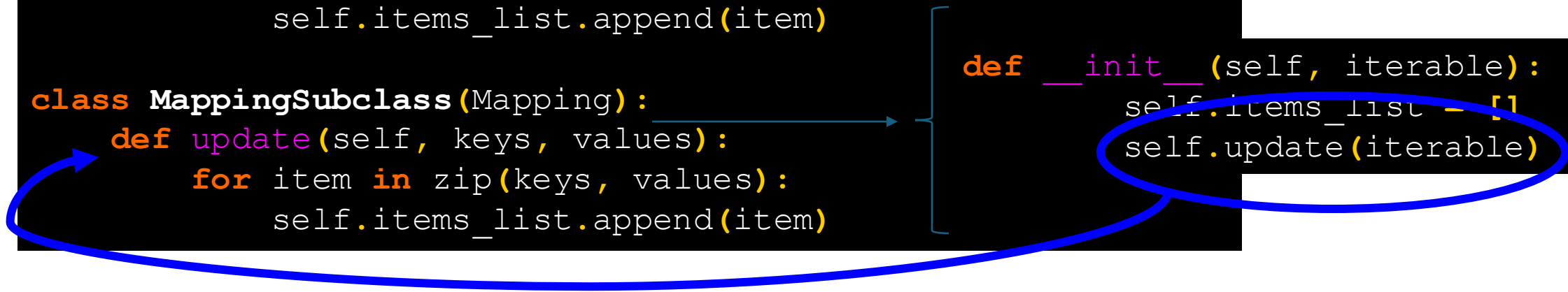
Name mangling

This `__init__` method references an `update` method. Python will simply look for the most local definition of `update` here.

```
class Mapping:
    def __init__(self, iterable):
        self.items_list = []
        self.update(iterable)
    def update(self, iterable):
        for item in iterable:
            self.items_list.append(item)
```

```
class MappingSubclass(Mapping):
    def update(self, keys, values):
        for item in zip(keys, values):
            self.items_list.append(item)
```

```
def __init__(self, iterable):
    self.items_list = []
    self.update(iterable)
```



Name mangling

The signatures of the update call and the update definition do not match. The `__init__` method depends on a certain implementation of update being available. Namely, the update defined in Mapping.

```
class Mapping:
    def __init__(self, iterable):
        self.items_list = []
        self.update(iterable)
    def update(self, iterable):
        for item in iterable:
            self.items_list.append(item)
```

```
class MappingSubclass(Mapping):
    def update(self, keys, values):
        for item in zip(keys, values):
            self.items_list.append(item)
```

```
def __init__(self, iterable):
    self.items_list = []
    self.update(iterable)
```



Name mangling

```
>>> import  
>>>      =      .      ([1, 2, 3])
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

File "map.py", line 4, in __init__

self.update(iterable)

TypeError: update() takes exactly 3 arguments (2 given)

Name mangling

```
class Mapping:
    def __init__(self, iterable):
        self.items_list = []
        self.__update(iterable)
    def update(self, iterable):
        for item in iterable:
            self.items_list.append(item)
    __update = update # private copy of original update() method

class MappingSubclass(Mapping):
    def update(self, keys, values):
        # provides new signature for update()
        # but does not break __init__()
        for item in zip(keys, values):
            self.items_list.append(item)
```

Name mangling

```
>>> import map
>>> x = map.MappingSubclass([1,2,3])
>>> x.items_list
[1, 2, 3]
>>> x.update(['key1', 'key2'], ['val1', 'val2'])
>>> x.items_list
[1, 2, 3, ('key1', 'val1'), ('key2', 'val2')]
```

Python & STRUCT

Structs in python

- You can create a struct-like object by using an empty class.

```
>>> class Struct:
...     pass
...
>>> node = Struct()
>>> node.label = 4
>>> node.data = "My data string"
>>> node.next = Struct()
>>> next_node = node.next
>>> next_node.label = 5
>>> print (node.next.label)
5
```

Emulating methods

- You can create custom classes that emulate methods that have significant meaning when combined with other Python objects.
- The statement `print >>` typically prints to the file-like object that follows. Specifically, the file-like object needs a `write()` method. This means I can make any class which, as long as it has a `write()` method, is a valid argument for this `print` statement.

```
>>> class Random:
...     def write(self, str_in):
...         print ("The string to write is: " + str(str_in))
>>> someobj = Random()
>>> print >> someobj, "whatever"
```

Polimorfism: Polymorphism with class methods

```
class P1:  
    def M1(self):  
        return 5  
  
class P2():  
    def M1(self):  
        return 10  
  
x1 = P1()  
x2 = P2()  
  
for x in (x1,x2):  
    print(x.M1())
```

Polymorphism with Inheritance

```
class Bird:
    def intro(self):
        print("There are many types of birds.")
    def flight(self):
        print("Most of the birds can fly but some cannot.")

class sparrow(Bird):
    def flight(self):
        print("Sparrows can fly.")

class ostrich(Bird):
    def flight(self):
        print("Ostriches cannot fly.")
```

```
obj_bird = Bird()
obj_spr = sparrow()
obj_ost = ostrich()
```

```
obj_bird.intro()
obj_bird.flight()
```

```
obj_spr.intro()
obj_spr.flight()
```

```
obj_ost.intro()
obj_ost.flight()
```